

PM1-AA1-EV02 — Taller: puesta en marcha del backend y verificación de los endpoints

Plan de Mejoramiento · Depuración y complemento de la app móvil de e-commerce

APRENDIZ Daniel Salas Roman	FICHA 3223873	PROGRAMA Análisis y Desarrollo de Software (228118)	CLIENTE HTTP curl
--------------------------------	------------------	---	----------------------

1. Puesta en marcha del backend

El backend es una API REST en Flask con SQLAlchemy sobre SQLite. Queda escuchando en `http://127.0.0.1:5050` y su terminal muestra cada petición que llega, que es justamente lo que la guía pide dejar visible para el diagnóstico.

Pasos ejecutados

1. Clonar el repositorio y crear el entorno virtual con `python -m venv venv`.
2. Instalar las dependencias. **Aquí apareció el primer obstáculo real:** el archivo se llama `requirement.txt` (en singular, no `requirements.txt`) e incluye `psycopg2-binary`, que sólo se usa en el despliegue y en Windows falla pidiendo `pg_config`. Como `pip` instala de forma atómica, ese único fallo deja **todo** sin instalar, y por eso después aparece `ModuleNotFoundError: No module named 'flask'` aunque el comando pareciera haber corrido. Se instalaron sólo las dependencias necesarias: Flask, Flask-SQLAlchemy, Flask-CORS, python-dotenv, PyJWT y Werkzeug.
3. Crear la base de datos ejecutando `database/create-ecommerce.sql`. Hay que borrar primero `database/ecommerce.db` si ya existe, porque el script usa `CREATE TABLE` sin `IF NOT EXISTS`. Queda con 5 países, 4 estados, 4 ciudades, 4 usuarios y 4 roles.
4. **Convertir a hash las contraseñas.** El script SQL las siembra en texto plano (`admin123`), pero `auth_routes.py` las valida con `check_password_hash` de Werkzeug. Un texto plano nunca coincide con un hash, así que el login responde 401 aunque la contraseña sea la correcta. Sin este paso el resto del taller es imposible.
5. Levantar el servidor con `python app.py`.

2. Tabla de endpoints probados

Las rutas se leyeron directamente del código del backend (`app/routes/`) y luego se probó cada una con `curl`. La columna «Token» indica si requiere la cabecera `Authorization: Bearer <token>`.

MÉTODO	RUTA	TOKEN	ESTRUCTURA QUE DEVUELVE	OBSERVACIÓN
GET	<code>/api/locations/countries</code>	No	<code>{"countries":[{"id_Country","CountryName"}],"count"}</code>	Llena el primer desplegable. Aquí estaba el bug del Reto 1.
GET	<code>/api/locations/states?country_id=1</code>	No	<code>{"states":[{"id_States","StatesName","id_Country","country"}],"count"}</code>	Estados del país elegido. Mismo envoltorio.
GET	<code>/api/locations/cities?state_id=1</code>	No	<code>{"cities":[{"id_City","CityName","id_States","state","country"}],"count"}</code>	Ciudades del estado elegido. Mismo envoltorio.
GET	<code>/api/locations/countries/{id}/states</code>	No	<code>{"country","states":[...],"count"}</code>	Ruta alterna que anida el país. La app no la usa.

MÉTODO	RUTA	TOKEN	ESTRUCTURA QUE DEVUELVE	OBSERVACIÓN
POST	/api/auth/login	No	{"message", "token", "user"}	Devuelve el JWT. Las llaves del cuerpo son Email y Passwordkey, con esa mayúscula exacta. Responde 401 si las credenciales fallan.
GET	/api/auth/roles	No	{"roles": [{ "idRole", "TypeRole" }]}	4 roles: Administrador, Usuario, Vendedor, Cliente. Bug del Reto 2.
GET	/api/users/profile	Sí	{"user": { "...", "city": { "...", "roles": [...] } }	Perfil del dueño del token. Bug del Reto 2. 401 sin token.
PUT	/api/users/profile	Sí	{"message", "user"}	Acepta UserName, id_City y Email. Responde 409 si el correo ya lo tiene otro usuario.
GET	/api/users/	Sí + Admin	{"users": [...], "pagination": { ... } }	Responde 403 si el rol no es Administrador.
PUT	/api/users/{id}/roles	Sí + Admin	{"message", "user"}	Cuerpo: { "role_ids": [3] } . 403 si no es Administrador.

3. El patrón que revela el diagnóstico

Al llenar esta tabla salta a la vista algo que no se ve mirando una sola ruta: **el backend nunca devuelve una lista pelada**. Siempre envuelve la colección dentro de un objeto con su llave y, casi siempre, un contador.

LO QUE EL APISERVICE ESPERABA

```
[
  { "id_Country": 1, "CountryName": "Colombia" },
  { "id_Country": 2, "CountryName": "Estados Unidos" }
]
```

LO QUE EL BACKEND DEVUELVE

```
{
  "countries": [
    { "id_Country": 1, "CountryName": "Colombia" },
    { "id_Country": 2, "CountryName": "Estados Unidos" }
  ],
  "count": 5
}
```

Esa diferencia entre `[]` y `{ }` es toda la causa del error. Gson lanza `JsonSyntaxException: Expected BEGIN_ARRAY but was BEGIN_OBJECT`, la excepción cae en el `catch` del ViewModel y el desplegable se queda vacío.

El corte más importante del diagnóstico

La guía lo plantea así: si el backend responde bien pero la app no muestra nada, el problema está del lado de Android. Esta tabla demuestra que **las diez rutas responden correctamente**, con los datos completos. Por lo tanto el fallo **no estaba en el servidor**, y eso descarta de una vez las hipótesis de «el endpoint no existe», «devuelve 404 o 500» y «la base está vacía».

4. Diferencia entre 401 y 403

Las pruebas dejaron los dos casos registrados, y conviene tenerlo claro porque es la base de la seguridad en dos capas del Reto 2:

- **401 Unauthorized** — *autenticación*. El servidor no sabe quién eres: no mandaste token, o el token no es válido o expiró. Se obtuvo pidiendo `/api/users/profile` sin cabecera.
- **403 Forbidden** — *autorización*. El servidor sí sabe quién eres, el token es válido, pero tu rol no alcanza para esa acción. Se obtuvo pidiendo `/api/users/` con el token de `juan_perez`, que no es Administrador.

5. Comandos exactos usados

```
curl -i http://127.0.0.1:5050/api/locations/countries

curl "http://127.0.0.1:5050/api/locations/states?country_id=1"

curl -X POST http://127.0.0.1:5050/api/auth/login \
-H "Content-Type: application/json" \
-d '{"Email":"admin@ecommerce.com","PasswoRDkey":"admin123"}'

curl -H "Authorization: Bearer $TOKEN" http://127.0.0.1:5050/api/users/profile
```

La salida completa de las trece pruebas está en el archivo `PM1-EV02-pruebas-endpoints.txt` y en la captura `PM1-EV02-captura-endpoints.png`, adjuntos a esta evidencia.